◆ Beginner Level String Problems (TypeScript)

```
// 1. Reverse a string
function reverseStringBuiltin(s: string): string {
  return s.split("").reverse().join("");
}

function reverseStringManual(s: string): string {
  let rev = "";
  for (let i = s.length - 1; i >= 0; i--) {
    rev += s[i];
  }
  return rev;
}


// 2. Check if string is palindrome
function isPalindromeBuiltin(s: string): boolean {
  return s === s.split("").reverse().join("");
}

function isPalindromeManual(s: string): boolean {
  let left = 0, right = s.length - 1;
  while (left < right) {
    if (s[left] !== s[right]) return false;
    left++; right--;
  }
  return true;
}


// 3. Find length of string
function stringLengthBuiltin(s: string): number {
  return s.length;
}

function stringLengthManual(s: string): number {
  let count = 0;
  for (let _ of s) count++;
  return count;
}


// 4. Count vowels and consonants
function countVCBuiltin(s: string): { vowels: number, consonants: number }
{
  const vowelsSet = new Set("aeiouAEIOU");
  let v = 0, c = 0;
  for (let ch of s) {
    if (/[a-zA-Z]/.test(ch)) {
      if (vowelsSet.has(ch)) v++;
      else c++;
    }
  }
  return { vowels: v, consonants: c };
}
```

```
function countVCManual(s: string): { vowels: number, consonants: number } {
  const vowels = "aeiouAEIOU";
  let v = 0, c = 0;
  for (let ch of s) {
    if ((ch >= "a" && ch <= "z") || (ch >= "A" && ch <= "Z")) {
      if (vowels.indexOf(ch) !== -1) v++;
      else c++;
    }
  }
  return { vowels: v, consonants: c };
}


// 5. Convert to uppercase/lowercase
function toUpperBuiltin(s: string): string {
  return s.toUpperCase();
}
function toLowerBuiltin(s: string): string {
  return s.toLowerCase();
}

function toUpperManual(s: string): string {
  let result = "";
  for (let ch of s) {
    if (ch >= "a" && ch <= "z") {
      result += String.fromCharCode(ch.charCodeAt(0) - 32);
    } else {
      result += ch;
    }
  }
  return result;
}

function toLowerManual(s: string): string {
  let result = "";
  for (let ch of s) {
    if (ch >= "A" && ch <= "Z") {
      result += String.fromCharCode(ch.charCodeAt(0) + 32);
    } else {
      result += ch;
    }
  }
  return result;
}


// 6. Remove whitespaces
function removeSpacesBuiltin(s: string): string {
  return s.replace(/\s+/g, "");
}

function removeSpacesManual(s: string): string {
  let result = "";
  for (let ch of s) {
    if (ch !== " ") result += ch;
  }
  return result;
}


// 7. Count frequency of each character
```

```
function charFreqBuiltin(s: string): Record<string, number> {
  const freq: Record<string, number> = {};
  for (let ch of s) freq[ch] = (freq[ch] || 0) + 1;
  return freq;
}

function charFreqManual(s: string): Record<string, number> {
  const freq: Record<string, number> = {};
  for (let i = 0; i < s.length; i++) {
    const ch = s[i];
    if (freq[ch] === undefined) freq[ch] = 1;
    else freq[ch]++;
  }
  return freq;
}


// 8. First non-repeating character
function firstNonRepeatBuiltin(s: string): string | null {
  const freq: Record<string, number> = {};
  for (let ch of s) freq[ch] = (freq[ch] || 0) + 1;
  for (let ch of s) if (freq[ch] === 1) return ch;
  return null;
}

function firstNonRepeatManual(s: string): string | null {
  for (let i = 0; i < s.length; i++) {
    let found = false;
    for (let j = 0; j < s.length; j++) {
      if (i !== j && s[i] === s[j]) {
        found = true;
        break;
      }
    }
    if (!found) return s[i];
  }
  return null;
}


// 9. Replace spaces with %20 (URLify)
function urlifyBuiltin(s: string): string {
  return s.replace(/ /g, "%20");
}

function urlifyManual(s: string): string {
  let result = "";
  for (let ch of s) {
    result += (ch === " ") ? "%20" : ch;
  }
  return result;
}


// 10. Compare two strings
function compareBuiltin(s1: string, s2: string): boolean {
  return s1 === s2;
}

function compareManual(s1: string, s2: string): boolean {
  if (s1.length !== s2.length) return false;
```

```
  for (let i = 0; i < s1.length; i++) {
    if (s1[i] !== s2[i]) return false;
  }
  return true;
}
```

◆ Substrings / Subsequences Problems (TypeScript)

```
// 11. Find all substrings of a string
function allSubstringsBuiltin(s: string): string[] {
  const result: string[] = [];
  for (let i = 0; i < s.length; i++) {
    for (let j = i + 1; j <= s.length; j++) {
      result.push(s.substring(i, j));
    }
  }
  return result;
}

function allSubstringsManual(s: string): string[] {
  const res: string[] = [];
  for (let i = 0; i < s.length; i++) {
    let sub = "";
    for (let j = i; j < s.length; j++) {
      sub += s[j];
      res.push(sub);
    }
  }
  return res;
}


// 12. Longest substring without repeating characters
function longestUniqueBuiltin(s: string): number {
  const seen = new Map<string, number>();
  let left = 0, maxLen = 0;
  for (let right = 0; right < s.length; right++) {
    if (seen.has(s[right]) && seen.get(s[right])! >= left) {
      left = seen.get(s[right])! + 1;
    }
    seen.set(s[right], right);
    maxLen = Math.max(maxLen, right - left + 1);
  }
  return maxLen;
}

function longestUniqueManual(s: string): number {
  let left = 0, maxLen = 0;
  const freq: Record<string, number> = {};
  for (let right = 0; right < s.length; right++) {
    const ch = s[right];
    if (freq[ch] === undefined) freq[ch] = 0;
    freq[ch]++;
    while (freq[ch] > 1) {
      freq[s[left]]--;
      left++;
    }
    maxLen = Math.max(maxLen, right - left + 1);
  }
  return maxLen;
}
```

```
// 13. Longest palindromic substring
function longestPalindromeBuiltin(s: string): string {
  let start = 0, maxLen = 1;
  function expand(l: number, r: number) {
    while (l >= 0 && r < s.length && s[l] === s[r]) {
      if (r - l + 1 > maxLen) {
        start = l;
        maxLen = r - l + 1;
      }
      l--; r++;
    }
  }
  for (let i = 0; i < s.length; i++) {
    expand(i, i);
    expand(i, i + 1);
  }
  return s.substring(start, start + maxLen);
}

function longestPalindromeManual(s: string): string {
  let best = "";
  function expand(l: number, r: number): string {
    while (l >= 0 && r < s.length && s[l] === s[r]) {
      l--; r++;
    }
    return s.slice(l + 1, r);
  }
  for (let i = 0; i < s.length; i++) {
    const p1 = expand(i, i);
    const p2 = expand(i, i + 1);
    const candidate = p1.length > p2.length ? p1 : p2;
    if (candidate.length > best.length) best = candidate;
  }
  return best;
}


// 14. Longest common prefix
function longestCommonPrefixBuiltin(strs: string[]): string {
  if (!strs.length) return "";
  return strs.reduce((a, b) => {
    let i = 0;
    while (i < a.length && i < b.length && a[i] === b[i]) i++;
    return a.slice(0, i);
  });
}

function longestCommonPrefixManual(strs: string[]): string {
  if (!strs.length) return "";
  let prefix = strs[0];
  for (let i = 1; i < strs.length; i++) {
    let j = 0;
    while (j < prefix.length && j < strs[i].length && prefix[j] ===
strs[i][j]) {
      j++;
    }
    prefix = prefix.slice(0, j);
  }
  return prefix;
}
```

```
// 15. Longest Common Subsequence (LCS)
function lcsBuiltin(s1: string, s2: string): number {
  const n = s1.length, m = s2.length;
  const dp = Array.from({ length: n + 1 }, () => Array(m + 1).fill(0));
  for (let i = 1; i <= n; i++) {
    for (let j = 1; j <= m; j++) {
      if (s1[i - 1] === s2[j - 1])
        dp[i][j] = dp[i - 1][j - 1] + 1;
      else
        dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
    }
  }
  return dp[n][m];
}

function lcsManual(s1: string, s2: string): number {
  return lcsBuiltin(s1, s2); // manual is same DP approach
}


// 16. Edit Distance (Levenshtein)
function editDistanceBuiltin(s1: string, s2: string): number {
  const n = s1.length, m = s2.length;
  const dp = Array.from({ length: n + 1 }, () => Array(m + 1).fill(0));
  for (let i = 0; i <= n; i++) dp[i][0] = i;
  for (let j = 0; j <= m; j++) dp[0][j] = j;
  for (let i = 1; i <= n; i++) {
    for (let j = 1; j <= m; j++) {
      if (s1[i - 1] === s2[j - 1]) dp[i][j] = dp[i - 1][j - 1];
      else dp[i][j] = 1 + Math.min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j
- 1]);
    }
  }
  return dp[n][m];
}

function editDistanceManual(s1: string, s2: string): number {
  return editDistanceBuiltin(s1, s2); // manual is same DP
}


// 17. Generate all permutations of a string
function permutationsBuiltin(s: string): string[] {
  if (s.length <= 1) return [s];
  const result: string[] = [];
  for (let i = 0; i < s.length; i++) {
    const char = s[i];
    const rest = s.slice(0, i) + s.slice(i + 1);
    for (const perm of permutationsBuiltin(rest)) {
      result.push(char + perm);
    }
  }
  return result;
}

function permutationsManual(s: string): string[] {
  return permutationsBuiltin(s); // recursion same logic
}
```

```ts
// 18. Generate all combinations of characters
function combinationsBuiltin(s: string): string[] {
  const res: string[] = [];
  function backtrack(start: number, path: string) {
    res.push(path);
    for (let i = start; i < s.length; i++) {
      backtrack(i + 1, path + s[i]);
    }
  }
  backtrack(0, "");
  return res;
}

function combinationsManual(s: string): string[] {
  return combinationsBuiltin(s); // backtracking is standard
}


// 19. Check if one string is subsequence of another
function isSubsequenceBuiltin(s1: string, s2: string): boolean {
  let i = 0;
  for (let ch of s2) if (i < s1.length && s1[i] === ch) i++;
  return i === s1.length;
}

function isSubsequenceManual(s1: string, s2: string): boolean {
  let i = 0, j = 0;
  while (i < s1.length && j < s2.length) {
    if (s1[i] === s2[j]) i++;
    j++;
  }
  return i === s1.length;
}


// 20. Smallest window containing all characters of another
function minWindowBuiltin(s: string, t: string): string {
  const need: Record<string, number> = {};
  for (let ch of t) need[ch] = (need[ch] || 0) + 1;
  let have: Record<string, number> = {};
  let required = Object.keys(need).length;
  let formed = 0, l = 0, ans: [number, number] = [-1, Infinity];

  for (let r = 0; r < s.length; r++) {
    let ch = s[r];
    have[ch] = (have[ch] || 0) + 1;
    if (have[ch] === need[ch]) formed++;
    while (l <= r && formed === required) {
      if (r - l < ans[1] - ans[0]) ans = [l, r];
      have[s[l]]--;
      if (have[s[l]] < need[s[l]]) formed--;
      l++;
    }
  }
  return ans[0] === -1 ? "" : s.slice(ans[0], ans[1] + 1);
}

function minWindowManual(s: string, t: string): string {
  return minWindowBuiltin(s, t); // sliding window is same idea
}
```

◆ Pattern Matching Problems (TypeScript)

```
// 21. Naive Pattern Matching
function naivePatternSearch(text: string, pattern: string): number[] {
  const res: number[] = [];
  for (let i = 0; i <= text.length - pattern.length; i++) {
    let j = 0;
    while (j < pattern.length && text[i + j] === pattern[j]) j++;
    if (j === pattern.length) res.push(i);
  }
  return res;
}


// 22. KMP Algorithm
function kmpSearch(text: string, pattern: string): number[] {
  const lps = buildLPS(pattern);
  const res: number[] = [];
  let i = 0, j = 0;
  while (i < text.length) {
    if (text[i] === pattern[j]) {
      i++; j++;
      if (j === pattern.length) {
        res.push(i - j);
        j = lps[j - 1];
      }
    } else {
      if (j !== 0) j = lps[j - 1];
      else i++;
    }
  }
  return res;
}

function buildLPS(pat: string): number[] {
  const lps = Array(pat.length).fill(0);
  let len = 0, i = 1;
  while (i < pat.length) {
    if (pat[i] === pat[len]) {
      len++; lps[i] = len; i++;
    } else {
      if (len !== 0) len = lps[len - 1];
      else { lps[i] = 0; i++; }
    }
  }
  return lps;
}


// 23. Rabin-Karp Algorithm
function rabinKarp(text: string, pattern: string, q = 101): number[] {
  const d = 256;
  const m = pattern.length, n = text.length;
  let p = 0, t = 0, h = 1;
  const res: number[] = [];

  for (let i = 0; i < m - 1; i++) h = (h * d) % q;

  for (let i = 0; i < m; i++) {
```

```
    p = (d * p + pattern.charCodeAt(i)) % q;
    t = (d * t + text.charCodeAt(i)) % q;
  }

  for (let i = 0; i <= n - m; i++) {
    if (p === t) {
      let match = true;
      for (let j = 0; j < m; j++) {
        if (text[i + j] !== pattern[j]) { match = false; break; }
      }
      if (match) res.push(i);
    }
    if (i < n - m) {
      t = (d * (t - text.charCodeAt(i) * h) + text.charCodeAt(i + m)) % q;
      if (t < 0) t += q;
    }
  }
  return res;
}


// 24. Z Algorithm
function zAlgorithm(s: string): number[] {
  const n = s.length;
  const Z = Array(n).fill(0);
  let l = 0, r = 0;
  for (let i = 1; i < n; i++) {
    if (i <= r) Z[i] = Math.min(r - i + 1, Z[i - l]);
    while (i + Z[i] < n && s[Z[i]] === s[i + Z[i]]) Z[i]++;
    if (i + Z[i] - 1 > r) { l = i; r = i + Z[i] - 1; }
  }
  return Z;
}


// 25. Find all occurrences of substring
function findAllOccurrencesBuiltin(text: string, pat: string): number[] {
  const res: number[] = [];
  let idx = text.indexOf(pat);
  while (idx !== -1) {
    res.push(idx);
    idx = text.indexOf(pat, idx + 1);
  }
  return res;
}

function findAllOccurrencesManual(text: string, pat: string): number[] {
  return naivePatternSearch(text, pat);
}


// 26. Check if string is rotation of another
function isRotationBuiltin(s1: string, s2: string): boolean {
  return s1.length === s2.length && (s1 + s1).includes(s2);
}

function isRotationManual(s1: string, s2: string): boolean {
  if (s1.length !== s2.length) return false;
  for (let i = 0; i < s1.length; i++) {
    const rotated = s1.slice(i) + s1.slice(0, i);
    if (rotated === s2) return true;
```

```
  }
  return false;
}


// 27. Check if two strings are anagrams
function areAnagramsBuiltin(s1: string, s2: string): boolean {
  return s1.split("").sort().join("") === s2.split("").sort().join("");
}

function areAnagramsManual(s1: string, s2: string): boolean {
  if (s1.length !== s2.length) return false;
  const count: Record<string, number> = {};
  for (let ch of s1) count[ch] = (count[ch] || 0) + 1;
  for (let ch of s2) {
    if (!count[ch]) return false;
    count[ch]--;
  }
  return true;
}


// 28. Group anagrams
function groupAnagramsBuiltin(strs: string[]): string[][] {
  const map = new Map<string, string[]>();
  for (let s of strs) {
    const key = s.split("").sort().join("");
    if (!map.has(key)) map.set(key, []);
    map.get(key)!.push(s);
  }
  return Array.from(map.values());
}

function groupAnagramsManual(strs: string[]): string[][] {
  const map: Record<string, string[]> = {};
  for (let s of strs) {
    const count = Array(26).fill(0);
    for (let ch of s) count[ch.charCodeAt(0) - 97]++;
    const key = count.join("#");
    if (!map[key]) map[key] = [];
    map[key].push(s);
  }
  return Object.values(map);
}


// 29. Check if two strings are isomorphic
function isIsomorphicBuiltin(s: string, t: string): boolean {
  if (s.length !== t.length) return false;
  const mapST: Record<string, string> = {};
  const mapTS: Record<string, string> = {};
  for (let i = 0; i < s.length; i++) {
    const a = s[i], b = t[i];
    if ((mapST[a] && mapST[a] !== b) || (mapTS[b] && mapTS[b] !== a))
return false;
    mapST[a] = b;
    mapTS[b] = a;
  }
  return true;
}
```

```
function isIsomorphicManual(s: string, t: string): boolean {
  return isIsomorphicBuiltin(s, t); // mapping logic same
}


// 30. Check if two strings are scrambled versions
function isScrambleBuiltin(s1: string, s2: string): boolean {
  if (s1 === s2) return true;
  if (s1.length !== s2.length) return false;
  if (s1.split("").sort().join("") !== s2.split("").sort().join("")) return
false;
  for (let i = 1; i < s1.length; i++) {
    if ((isScrambleBuiltin(s1.slice(0, i), s2.slice(0, i)) &&
         isScrambleBuiltin(s1.slice(i), s2.slice(i))) ||
        (isScrambleBuiltin(s1.slice(0, i), s2.slice(-i)) &&
         isScrambleBuiltin(s1.slice(i), s2.slice(0, s2.length - i)))) {
      return true;
    }
  }
  return false;
}

function isScrambleManual(s1: string, s2: string): boolean {
  return isScrambleBuiltin(s1, s2); // recursive DP standard
}
```

◆ Advanced String Manipulations (TypeScript)

```
// 31. Longest repeating substring
function longestRepeatingSubstringBuiltin(s: string): string {
  let longest = "";
  for (let i = 0; i < s.length; i++) {
    for (let j = i + 1; j <= s.length; j++) {
      const sub = s.slice(i, j);
      if (s.indexOf(sub, i + 1) !== -1 && sub.length > longest.length) {
        longest = sub;
      }
    }
  }
  return longest;
}

function longestRepeatingSubstringManual(s: string): string {
  // Same logic, brute force, without slice
  let longest = "";
  for (let i = 0; i < s.length; i++) {
    let sub = "";
    for (let j = i; j < s.length; j++) {
      sub += s[j];
      if (s.indexOf(sub, i + 1) !== -1 && sub.length > longest.length) {
        longest = sub;
      }
    }
  }
  return longest;
}


// 32. Lexicographically smallest and largest substring of length k
function smallestLargestSubstringBuiltin(s: string, k: number): { smallest:
string, largest: string } {
  let smallest = s.slice(0, k);
  let largest = s.slice(0, k);
  for (let i = 1; i <= s.length - k; i++) {
    const sub = s.slice(i, i + k);
    if (sub < smallest) smallest = sub;
    if (sub > largest) largest = sub;
  }
  return { smallest, largest };
}

function smallestLargestSubstringManual(s: string, k: number) {
  let smallest = s.substring(0, k);
  let largest = s.substring(0, k);
  for (let i = 1; i <= s.length - k; i++) {
    let sub = "";
    for (let j = 0; j < k; j++) sub += s[i + j];
    if (sub < smallest) smallest = sub;
    if (sub > largest) largest = sub;
  }
```

```
  return { smallest, largest };
}


// 33. Minimum bracket reversals to balance
function minBracketReversals(s: string): number {
  if (s.length % 2 !== 0) return -1;
  let open = 0, close = 0;
  for (let ch of s) {
    if (ch === '{') open++;
    else {
      if (open === 0) close++;
      else open--;
    }
  }
  return Math.ceil(open / 2) + Math.ceil(close / 2);
}


// 34. Validate balanced parentheses
function isBalancedBuiltin(s: string): boolean {
  const stack: string[] = [];
  const map: Record<string, string> = { ")": "(", "}": "{", "]": "[" };
  for (let ch of s) {
    if (["(", "{", "["].includes(ch)) stack.push(ch);
    else if (map[ch]) {
      if (stack.pop() !== map[ch]) return false;
    }
  }
  return stack.length === 0;
}

function isBalancedManual(s: string): boolean {
  const stack: string[] = [];
  for (let i = 0; i < s.length; i++) {
    const ch = s[i];
    if (ch === "(" || ch === "{" || ch === "[") stack.push(ch);
    else {
      if (!stack.length) return false;
      const top = stack.pop()!;
      if ((ch === ")" && top !== "(") || (ch === "}" && top !== "{") || (ch
=== "]" && top !== "[")) {
        return false;
      }
    }
  }
  return stack.length === 0;
}


// 35. Remove adjacent duplicates
function removeAdjacentDuplicatesBuiltin(s: string): string {
  const stack: string[] = [];
  for (let ch of s) {
    if (stack.length && stack[stack.length - 1] === ch) stack.pop();
    else stack.push(ch);
  }
  return stack.join("");
}

function removeAdjacentDuplicatesManual(s: string): string {
```

```
  let result = "";
  for (let i = 0; i < s.length; i++) {
    if (result.length && result[result.length - 1] === s[i]) {
      result = result.slice(0, -1);
    } else {
      result += s[i];
    }
  }
  return result;
}


// 36. Run Length Encoding
function runLengthEncodeBuiltin(s: string): string {
  let encoded = "";
  let count = 1;
  for (let i = 1; i <= s.length; i++) {
    if (s[i] === s[i - 1]) count++;
    else {
      encoded += s[i - 1] + count;
      count = 1;
    }
  }
  return encoded;
}

function runLengthDecodeBuiltin(s: string): string {
  let decoded = "";
  for (let i = 0; i < s.length; i += 2) {
    decoded += s[i].repeat(Number(s[i + 1]));
  }
  return decoded;
}


// 37. Implement atoi()
function myAtoiBuiltin(s: string): number {
  return parseInt(s, 10) || 0;
}

function myAtoiManual(s: string): number {
  let num = 0, sign = 1, i = 0;
  if (s[0] === "-") { sign = -1; i = 1; }
  for (; i < s.length; i++) {
    const code = s.charCodeAt(i) - 48;
    if (code < 0 || code > 9) break;
    num = num * 10 + code;
  }
  return num * sign;
}


// 38. Implement strstr()
function strstrBuiltin(haystack: string, needle: string): number {
  return haystack.indexOf(needle);
}

function strstrManual(haystack: string, needle: string): number {
  for (let i = 0; i <= haystack.length - needle.length; i++) {
    let j = 0;
    while (j < needle.length && haystack[i + j] === needle[j]) j++;
```

```
    if (j === needle.length) return i;
  }
  return -1;
}


// 39. Integer to string (itoa)
function myItoaBuiltin(num: number): string {
  return num.toString();
}

function myItoaManual(num: number): string {
  if (num === 0) return "0";
  let str = "", n = Math.abs(num);
  while (n > 0) {
    str = String.fromCharCode((n % 10) + 48) + str;
    n = Math.floor(n / 10);
  }
  return num < 0 ? "-" + str : str;
}


// 40. Find longest word in a sentence
function longestWordBuiltin(sentence: string): string {
  return sentence.split(" ").reduce((a, b) => (b.length > a.length ? b :
a), "");
}

function longestWordManual(sentence: string): string {
  let longest = "", word = "";
  for (let ch of sentence + " ") {
    if (ch !== " ") word += ch;
    else {
      if (word.length > longest.length) longest = word;
      word = "";
    }
  }
  return longest;
}
```

◆ Palindrome-based String Problems (TypeScript)

```
// 41. Find all palindromic substrings
function allPalindromicSubstringsBuiltin(s: string): string[] {
  const res: string[] = [];
  for (let i = 0; i < s.length; i++) {
    for (let j = i + 1; j <= s.length; j++) {
      const sub = s.slice(i, j);
      if (sub === sub.split("").reverse().join("")) res.push(sub);
    }
  }
  return res;
}

function allPalindromicSubstringsManual(s: string): string[] {
  const res: string[] = [];
  function expandAroundCenter(l: number, r: number) {
    while (l >= 0 && r < s.length && s[l] === s[r]) {
      res.push(s.slice(l, r + 1));
      l--; r++;
    }
  }
  for (let i = 0; i < s.length; i++) {
    expandAroundCenter(i, i);     // odd length
    expandAroundCenter(i, i + 1); // even length
  }
  return res;
}


// 42. Count palindromic substrings
function countPalindromicSubstringsBuiltin(s: string): number {
  return allPalindromicSubstringsBuiltin(s).length;
}

function countPalindromicSubstringsManual(s: string): number {
  let count = 0;
  function expand(l: number, r: number) {
    while (l >= 0 && r < s.length && s[l] === s[r]) {
      count++;
      l--; r++;
    }
  }
  for (let i = 0; i < s.length; i++) {
    expand(i, i);     // odd length
    expand(i, i + 1); // even length
  }
  return count;
}


// 43. Check if string can be rearranged into a palindrome
function canRearrangePalindromeBuiltin(s: string): boolean {
  const freq: Record<string, number> = {};
```

```
  for (let ch of s) freq[ch] = (freq[ch] || 0) + 1;
  let oddCount = 0;
  for (let v of Object.values(freq)) if (v % 2 !== 0) oddCount++;
  return oddCount <= 1;
}

function canRearrangePalindromeManual(s: string): boolean {
  const freq: Record<string, number> = {};
  for (let i = 0; i < s.length; i++) {
    const ch = s[i];
    freq[ch] = (freq[ch] || 0) + 1;
  }
  let oddCount = 0;
  for (let key in freq) {
    if (freq[key] % 2 !== 0) oddCount++;
  }
  return oddCount <= 1;
}


// 44. Minimum insertions to make string palindrome
function minInsertionsPalindrome(s: string): number {
  const n = s.length;
  const dp: number[][] = Array.from({ length: n }, () => Array(n).fill(0));
  for (let len = 2; len <= n; len++) {
    for (let i = 0; i <= n - len; i++) {
      let j = i + len - 1;
      if (s[i] === s[j]) dp[i][j] = dp[i + 1][j - 1];
      else dp[i][j] = 1 + Math.min(dp[i + 1][j], dp[i][j - 1]);
    }
  }
  return dp[0][n - 1];
}


// 45. Palindrome partitioning (min cuts)
function minCutPalindromePartition(s: string): number {
  const n = s.length;
  const cut = Array(n).fill(0);
  const pal: boolean[][] = Array.from({ length: n }, () =>
Array(n).fill(false));

  for (let i = 0; i < n; i++) {
    let minCut = i;
    for (let j = 0; j <= i; j++) {
      if (s[i] === s[j] && (i - j < 2 || pal[j + 1][i - 1])) {
        pal[j][i] = true;
        minCut = j === 0 ? 0 : Math.min(minCut, cut[j - 1] + 1);
      }
    }
    cut[i] = minCut;
  }
  return cut[n - 1];
}
```

◆ Special Interview Problems (TypeScript)

```
// 46. Longest substring with k unique characters
function longestSubstringKUniqueBuiltin(s: string, k: number): string {
  let left = 0, maxLen = 0, start = 0;
  const map: Record<string, number> = {};
  for (let right = 0; right < s.length; right++) {
    map[s[right]] = (map[s[right]] || 0) + 1;
    while (Object.keys(map).length > k) {
      map[s[left]]--;
      if (map[s[left]] === 0) delete map[s[left]];
      left++;
    }
    if (right - left + 1 > maxLen) {
      maxLen = right - left + 1;
      start = left;
    }
  }
  return s.substring(start, start + maxLen);
}

function longestSubstringKUniqueManual(s: string, k: number): string {
  // Same sliding window but counting manually
  let left = 0, maxLen = 0, start = 0;
  const freq: { [key: string]: number } = {};
  let uniqueCount = 0;

  for (let right = 0; right < s.length; right++) {
    if (!freq[s[right]]) uniqueCount++;
    freq[s[right]] = (freq[s[right]] || 0) + 1;

    while (uniqueCount > k) {
      freq[s[left]]--;
      if (freq[s[left]] === 0) {
        delete freq[s[left]];
        uniqueCount--;
      }
      left++;
    }

    if (right - left + 1 > maxLen) {
      maxLen = right - left + 1;
      start = left;
    }
  }
  return s.slice(start, start + maxLen);
}


// 47. Remove all duplicate characters
function removeDuplicatesBuiltin(s: string): string {
  return [...new Set(s)].join("");
}

function removeDuplicatesManual(s: string): string {
```

```
  let seen: Record<string, boolean> = {};
  let res = "";
  for (let ch of s) {
    if (!seen[ch]) {
      seen[ch] = true;
      res += ch;
    }
  }
  return res;
}


// 48. Smallest substring containing all unique characters
function smallestSubstringAllUniqueBuiltin(s: string): string {
  const uniqueChars = new Set(s);
  const required = uniqueChars.size;
  let left = 0, formed = 0;
  const freq: Record<string, number> = {};
  let ans = [Infinity, 0, 0];

  for (let right = 0; right < s.length; right++) {
    freq[s[right]] = (freq[s[right]] || 0) + 1;
    if (freq[s[right]] === 1) formed++;

    while (formed === required) {
      if (right - left + 1 < ans[0]) ans = [right - left + 1, left, right];
      freq[s[left]]--;
      if (freq[s[left]] === 0) formed--;
      left++;
    }
  }
  return ans[0] === Infinity ? "" : s.slice(ans[1], ans[2] + 1);
}

function smallestSubstringAllUniqueManual(s: string): string {
  return smallestSubstringAllUniqueBuiltin(s); // same sliding window logic
}


// 49. Find the first repeating character
function firstRepeatingCharBuiltin(s: string): string | null {
  const seen = new Set<string>();
  for (let ch of s) {
    if (seen.has(ch)) return ch;
    seen.add(ch);
  }
  return null;
}

function firstRepeatingCharManual(s: string): string | null {
  let seen: { [ch: string]: boolean } = {};
  for (let i = 0; i < s.length; i++) {
    const ch = s[i];
    if (seen[ch]) return ch;
    seen[ch] = true;
  }
  return null;
}


// 50. Next lexicographical permutation
```

```
function nextPermutationBuiltin(s: string): string {
  let arr = s.split("");
  let i = arr.length - 2;
  while (i >= 0 && arr[i] >= arr[i + 1]) i--;
  if (i >= 0) {
    let j = arr.length - 1;
    while (arr[j] <= arr[i]) j--;
    [arr[i], arr[j]] = [arr[j], arr[i]];
  }
  let left = i + 1, right = arr.length - 1;
  while (left < right) {
    [arr[left], arr[right]] = [arr[right], arr[left]];
    left++; right--;
  }
  return arr.join("");
}

function nextPermutationManual(s: string): string {
  return nextPermutationBuiltin(s); // manual swap & reverse logic same
}


// 51. Wildcard pattern matching (?, *)
function wildcardMatchBuiltin(s: string, p: string): boolean {
  const regex = new RegExp("^" + p.replace(/\*/g, ".*").replace(/\?/g, ".")
+ "$");
  return regex.test(s);
}

function wildcardMatchManual(s: string, p: string): boolean {
  const dp: boolean[][] = Array.from({ length: s.length + 1 }, () =>
    Array(p.length + 1).fill(false)
  );
  dp[0][0] = true;

  for (let j = 1; j <= p.length; j++) {
    if (p[j - 1] === "*") dp[0][j] = dp[0][j - 1];
  }

  for (let i = 1; i <= s.length; i++) {
    for (let j = 1; j <= p.length; j++) {
      if (p[j - 1] === "?" || p[j - 1] === s[i - 1]) dp[i][j] = dp[i - 1][j
- 1];
      else if (p[j - 1] === "*") dp[i][j] = dp[i][j - 1] || dp[i - 1][j];
    }
  }
  return dp[s.length][p.length];
}


// 52. Regex matching (., *)
function regexMatchBuiltin(s: string, p: string): boolean {
  return new RegExp("^" + p + "$").test(s);
}

function regexMatchManual(s: string, p: string): boolean {
  const dp: boolean[][] = Array.from({ length: s.length + 1 }, () =>
    Array(p.length + 1).fill(false)
  );
  dp[0][0] = true;
```

```
  for (let j = 1; j <= p.length; j++) {
    if (p[j - 1] === "*" && j >= 2) dp[0][j] = dp[0][j - 2];
  }

  for (let i = 1; i <= s.length; i++) {
    for (let j = 1; j <= p.length; j++) {
      if (p[j - 1] === "." || p[j - 1] === s[i - 1]) dp[i][j] = dp[i - 1][j
- 1];
      else if (p[j - 1] === "*") {
        dp[i][j] = dp[i][j - 2];
        if (p[j - 2] === "." || p[j - 2] === s[i - 1]) {
          dp[i][j] = dp[i][j] || dp[i - 1][j];
        }
      }
    }
  }
  return dp[s.length][p.length];
}


// 53. Longest duplicate substring
function longestDuplicateSubstringBuiltin(s: string): string {
  let longest = "";
  for (let i = 0; i < s.length; i++) {
    for (let j = i + 1; j <= s.length; j++) {
      const sub = s.slice(i, j);
      if (sub.length > longest.length && s.indexOf(sub, i + 1) !== -1) {
        longest = sub;
      }
    }
  }
  return longest;
}

function longestDuplicateSubstringManual(s: string): string {
  return longestDuplicateSubstringBuiltin(s); // naive search
}


// 54. Reverse words in a sentence
function reverseWordsBuiltin(s: string): string {
  return s.trim().split(/\s+/).reverse().join(" ");
}

function reverseWordsManual(s: string): string {
  let words: string[] = [];
  let word = "";
  for (let ch of s + " ") {
    if (ch !== " ") word += ch;
    else if (word) {
      words.push(word);
      word = "";
    }
  }
  let result = "";
  for (let i = words.length - 1; i >= 0; i--) {
    result += words[i];
    if (i > 0) result += " ";
  }
  return result;
}
```

```
// 55. Check if one string is a subsequence of another
function isSubsequenceBuiltin(s: string, t: string): boolean {
  let i = 0, j = 0;
  while (i < s.length && j < t.length) {
    if (s[i] === t[j]) i++;
    j++;
  }
  return i === s.length;
}

function isSubsequenceManual(s: string, t: string): boolean {
  let si = 0;
  for (let tj = 0; tj < t.length && si < s.length; tj++) {
    if (t[tj] === s[si]) si++;
  }
  return si === s.length;
}
```

.

◆ Miscellaneous String Problems (TypeScript)

```
// 56. Check if two strings are rotations of each other
function areRotationsBuiltin(s1: string, s2: string): boolean {
  if (s1.length !== s2.length) return false;
  return (s1 + s1).includes(s2);
}

function areRotationsManual(s1: string, s2: string): boolean {
  if (s1.length !== s2.length) return false;
  for (let i = 0; i < s1.length; i++) {
    let rotated = "";
    for (let j = 0; j < s1.length; j++) {
      rotated += s1[(i + j) % s1.length];
    }
    if (rotated === s2) return true;
  }
  return false;
}


// 57. Check if string is numeric
function isNumericBuiltin(s: string): boolean {
  return /^-?\d+(\.\d+)?$/.test(s.trim());
}

function isNumericManual(s: string): boolean {
  if (!s.length) return false;
  let hasDecimal = false, start = 0;
  if (s[0] === "-" || s[0] === "+") start = 1;
  for (let i = start; i < s.length; i++) {
    const ch = s[i];
    if (ch === ".") {
      if (hasDecimal) return false;
      hasDecimal = true;
    } else if (ch < "0" || ch > "9") {
      return false;
    }
  }
  return true;
}


// 58. Shortest string containing two strings as subsequences
function shortestCommonSupersequenceBuiltin(str1: string, str2: string):
string {
  const m = str1.length, n = str2.length;
  const dp: number[][] = Array.from({ length: m + 1 }, () => Array(n +
1).fill(0));

  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
```

```
      if (str1[i - 1] === str2[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
      else dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
    }
  }

  let i = m, j = n, result = "";
  while (i > 0 && j > 0) {
    if (str1[i - 1] === str2[j - 1]) {
      result = str1[i - 1] + result;
      i--; j--;
    } else if (dp[i - 1][j] > dp[i][j - 1]) {
      result = str1[i - 1] + result;
      i--;
    } else {
      result = str2[j - 1] + result;
      j--;
    }
  }
  while (i > 0) { result = str1[i - 1] + result; i--; }
  while (j > 0) { result = str2[j - 1] + result; j--; }
  return result;
}

function shortestCommonSupersequenceManual(str1: string, str2: string):
string {
  return shortestCommonSupersequenceBuiltin(str1, str2); // manual LCS
build is same logic
}


// 59. Word Break Problem
function wordBreakBuiltin(s: string, wordDict: string[]): boolean {
  const wordSet = new Set(wordDict);
  const dp: boolean[] = Array(s.length + 1).fill(false);
  dp[0] = true;

  for (let i = 1; i <= s.length; i++) {
    for (let j = 0; j < i; j++) {
      if (dp[j] && wordSet.has(s.substring(j, i))) {
        dp[i] = true;
        break;
      }
    }
  }
  return dp[s.length];
}

function wordBreakManual(s: string, wordDict: string[]): boolean {
  return wordBreakBuiltin(s, wordDict); // same DP approach
}


// 60. Print all valid IP addresses from a string
function restoreIpAddressesBuiltin(s: string): string[] {
  const res: string[] = [];
  function backtrack(start: number, parts: string[]) {
    if (parts.length === 4 && start === s.length) {
      res.push(parts.join("."));
      return;
    }
    if (parts.length === 4) return;
```

```
    for (let len = 1; len <= 3; len++) {
      if (start + len > s.length) break;
      const part = s.substring(start, start + len);
      if ((part.startsWith("0") && part.length > 1) || parseInt(part) >
255) continue;
      backtrack(start + len, [...parts, part]);
    }
  }
  backtrack(0, []);
  return res;
}

function restoreIpAddressesManual(s: string): string[] {
  return restoreIpAddressesBuiltin(s); // backtracking logic same
}
```
